

KanaReader – Push to GitHub & Deploy Guide

Date: 2025-08-21

Purpose

This document records the exact steps used to push the KanaReader project to GitHub and get it running online. It includes preflight checks, the Git commands executed, how to enable hosting, and how to update or roll back.

0) Preflight (quick checks)

- Ensure links and fetches are relative (works under a GitHub Pages subpath). Example: use `fetch('stories.json')` — not `fetch('/stories.json')`.
- Meta tags for mobile rendering added to *index.html*: `<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">` and `<meta name="theme-color" content="#111111">`.
- Inputs configured to be IME-friendly (romaji→kana and no autocorrect): `inputmode="text" autocomplete="off" autocapitalize="off" autocorrect="off" spellcheck="false"`.
- One user-initiated button exists for audio/TTS (mobile browsers block autoplay without a tap).
- Speech recognition / TTS gracefully degrade: mic button hidden/disabled if unsupported; rest of UI still works.

Console sanity-check used for relative fetch

```
// In DevTools Console, confirm the URL resolves under your current folder
new URL('stories.json', document.baseURI).href
// Expected: https://<host>/<repo>/stories.json (not the root /stories.json)

// Basic fetch test
fetch('stories.json')
  .then(r => { console.log('URL:', r.url, 'status:', r.status, 'ok:', r.ok); return r.text(); })
  .then(txt => console.log('First 120 chars:\n', txt.slice(0, 120)));
```

1) Install & configure Git (once per machine)

Installed Git for Windows and verified the version, then set user identity.

```
git --version
```

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

2) Create the repository and first commit

From the project root folder in PowerShell:

```
# Initialize and switch to the main branch
git init
git checkout -b main
```

```
# .gitignore (optional but recommended)
```

```
# node_modules, logs, OS cruft, and build outputs
echo "node_modules/
.DS_Store
*.log
dist/
" > .gitignore

# Stage and commit all files
git add -A
git commit -m "feat: first public lesson build (mobile-ready)"
```

3) Create the GitHub repo and push

Create an empty repository on GitHub (no README). Then connect and push:

```
# Replace with your repo URL (HTTPS used here)
git remote add origin https://github.com/learntospeak/langylearner.git
git branch -M main
git push -u origin main
```

4) Hosting options

Option A — GitHub Pages (simplest for static hosting):

- Make the repo public (required for GitHub Pages on personal accounts).
- GitHub → Repository → Settings → Pages → Build and deployment: Source = “Deploy from a branch”, Branch = “main / (root)”. Save.
- Wait 1–3 minutes for the site to appear at: <https://.github.io/>
- If the app assumed absolute paths (e.g., /stories.json), switch to relative paths or adjust with a <base> tag.

Option B — Vercel/Netlify (good previews and rollbacks):

Connect the GitHub repo, choose a static project (no build), and deploy. Each push to main creates a new preview/live build.

5) Update flow (after edits)

```
git add -A
git commit -m "chore: tweaks / fixes"
git push
```

For easy rollback, tag releases:

```
git tag -a v0.1.0 -m "public beta: greetings lesson"
git push --tags
```

6) Troubleshooting quick reference

- **Pages screen has no deployment controls:** repository is private. Make it public to use GitHub Pages on personal accounts.
- **404 after deploy:** ensure links/fetches are relative. Example: `fetch('stories.json')` (not `fetch('/stories.json')`).

- **Site loads but audio doesn't play:** on mobile, TTS must be user-initiated. Tap a "Play/Listen" button before speaking.
- **Speech recognition doesn't work on iOS:** Safari on iOS doesn't expose in-browser recognition. Show fallback UI.
- **Blank page:** open DevTools → Console for errors. Look for path mistakes or missing files.

Notes specific to this project

- WanaKana bindings and the smart normalizer are attached once per input to avoid double-handling key events.
- Cloze and translate steps accept kana readings for certain kanji (e.g., 一 → 一).
- Period/。 handling is ignored during checks so final punctuation doesn't block correctness.
- All fetches use relative paths (e.g., `fetch('stories.json')`) for subpath hosting.

Repository

<https://github.com/learntospeak/langylearner>